作业 HW2 栈和队列实验报告

姓名：施沂邑 学号：2553601 日期：2026年10月18日

本次实验完成列车进站、布尔表达式求值、最长合法括号子串、矩阵区域计数和队列最大值五项任务。前三题利用栈保存暂存元素、运算状态或下标，后两题利用队列完成搜索和最大值维护。报告按照模板分别说明问题、实现方法、调试过程和实验体会。

1 涉及数据结构和相关背景

栈限制元素在同一端插入和删除，具有后进先出的特点；队列在队尾插入、队首删除，具有先进先出的特点。双端队列允许在两端操作，适合维护单调的候选序列。本实验使用 C++ 标准库容器实现这些操作，重点是选择保存的信息以及确定何时入栈、出栈或入队、出队。

题目	主要结构	保存的信息
列车进站	stack<char>	暂时停放的列车
布尔表达式	两个 stack<char>	操作数与运算符
最长括号子串	stack<int>	未匹配左括号下标和边界
矩阵区域计数	queue<pair<int,int>>	已发现但未检查的格子坐标
队列中的最大值	queue<int>、deque<int>	完整队列与最大值候选

实验环境为 Windows、Visual Studio Code 和 MSYS2 UCRT64 g++ 15.2.0。复验使用 C++17、-O2 和标准库断言。源文件分别为 hw2_1.cpp 至 hw2_5.cpp。五题均以 main() 组织输入、状态更新和输出，表达式题另设求值函数，矩阵题另设坐标检查函数。

2 实验内容

每题均按“问题描述、基本要求、数据结构设计、功能说明、调试分析、总结和体会”展开。调试以题目提供的数据生成程序和参考判定逻辑为依据；样例、补充边界检查与生成程序批测分别记录。生成文件用于本地复验，不等同于在线评测的隐藏数据。

2.1 列车进站

2.1.1 问题描述

给定列车到达入口的顺序，判断若干目标出站顺序能否通过直接出站或在单轨车站中暂存实现。车站中的列车只能后进先出。

2.1.2 基本要求

第一行给出入站序列，后续各行给出待判断的出站序列，读至 EOF。可实现时输出 yes，否则输出 no。先检查两个序列长度是否相等；按题目的列车编号互不相同的通常约定进行模拟，不依赖字母表顺序。

2.1.3 数据结构设计

station 保存暂存列车；i 指向下一辆需要出站的车，j 指向入口中下一辆待处理的车。优先匹配栈顶，否则处理入口列车；入口耗尽且仍无法匹配时停止。

2.1.4 功能说明

main() 逐组读取目标序列，每组重新建立栈。下面为单组判断的核心实现，省略外层输入和长度检查。单组时间 O(L)，辅助空间 O(L)。

核心代码 1-1 单组列车进出站模拟

```
stack<char> station;
int i=0;
int j=0;
while(i<seq.size()) {
  if(!station.empty()&&station.top()==seq[i]) {
    station.pop();
    i++;
  } else if(j<str.size()) {
    if(seq[i]!=str[j]) {
      station.push(str[j++]);
    } else {
      j++;
      i++;
    }
  } else {
    break;
  }
}
if(i==seq.size()) {
  cout<<"yes"<<endl;
} else {
  cout<<"no"<<endl;
}
```

2.1.5 调试分析

处理 cab 时曾出现死循环：c 输出后需要 a，但栈顶是 b，且入口已耗尽。增加最后的 else 和 break，使失败分支能够退出。题面 abc 的六种目标排列复验结果为 yes、yes、yes、yes、no、yes，与样例一致。

2.1.6 总结和体会

入站顺序与出栈顺序不同，列车可以交替进出，不能只把整个输入倒序。编写模拟循环时，要检查每个分支是否推进状态或退出，避免遇到不可实现的目标时陷入死循环。

2.2 布尔表达式

2.2.1 问题描述

计算由 V、F、!、&、| 和括号组成的合法布尔表达式。V 表示真，F 表示假，运算优先级依次为 !、&、|，括号改变计算次序。

2.2.2 基本要求

每行一个表达式，符号之间可能有空格，按行读取并跳过空格。输出格式为 Expression k: V 或 Expression k: F，冒号后保留一个空格，结果各占一行。

2.2.3 数据结构设计

operands 保存布尔值，operators 保存运算符和左括号。按 !、&、| 的优先级执行运算；遇右括号先计算括号内的表达式。连续 ! 先入栈，等操作数读入后再处理。

2.2.4 功能说明

caculation() 按引用接收两个栈，无返回值：&、| 取两个操作数，! 取一个操作数，结果回栈并移除运算符。随后给出 main() 中的扫描部分。省略输入、编号和输出，保留全部运算分支。时间、空间均为 O(L)。

核心代码 2-1 单次布尔运算函数

```
void caculation(stack<char>& operators, stack<char>& operands) {
  if(operators.top()=='&') {
    char op1=operands.top();
    operands.pop();
    char op2=operands.top();
    operands.pop();
    if(op1=='F'||op2=='F') {
      operands.push('F');
    } else{
      operands.push('V');
    }
  } else if(operators.top()=='|') {
    char op1=operands.top();
    operands.pop();
    char op2=operands.top();
    operands.pop();
    if(op1=='V'||op2=='V') {
      operands.push('V');
    } else{
      operands.push('F');
    }
```

核心代码 2-1 续

```
  } else if(operators.top()=='!') {
    if(operands.top()=='F') {
      operands.pop();
      operands.push('V');
    } else {
      operands.pop();
      operands.push('F');
    }
  }
  operators.pop();
}
```

核心代码 2-2 主程序中的表达式扫描与收尾

```
for(int i=0; i<str.size(); i++) {
  if(str[i]==' ') {
    continue;
  } else if(str[i]=='F'||str[i]=='V') {
    operands.push(str[i]);
  } else if(str[i]==')') {
    while(operators.top()!='(') {
      caculation(operators, operands);
    }
    operators.pop();
  } else if(str[i]=='('||str[i]=='!') {
    operators.push(str[i]);
 } else if(str[i]=='&') {
    while(!operators.empty()&&operators.top()=='!') {
      caculation(operators, operands);
    }
    operators.push(str[i]);
 } else {
    while(!operators.empty()&&(operators.top()=='!'||operators.top()=='&')) {
      caculation(operators, operands);
    }
    operators.push(str[i]);
 }
}
while(!operators.empty()) {
  caculation(operators, operands);
}
```

2.2.5 调试分析

曾把运算符压入 operands，造成 operators 为空时访问 top()；又把 != 误写成 ! 与 == 的组合。纠正容器和比较式后，补上输出空格，并使空白行不占编号。随机测试可由题面 96.py 生成；本次复验3条样例和3条补充表达式，均正确。

2.2.6 总结和体会

两个栈分别保存“值”和“尚未完成的运算”，名称和使用位置必须一致。top() 本身不会检查算法状态，调用前要确认栈不为空或由合法表达式的不变式保证其非空。

2.3 最长子串

2.3.1 问题描述

求只含左右括号的字符串中最长合法连续子串，输出长度和从0开始的起点。同长时取最早起点，无合法子串输出 0 0。

2.3.2 基本要求

长度0至100000，单组输入。getline 读取一次，空行和空文件均计算为空串。

2.3.3 数据结构设计

stack<int> 保存下标，栈底 -1 为初始边界。右括号匹配后，长度为 i-st.top()，起点为 st.top()+1；失配右括号替换边界。

2.3.4 功能说明

main() 的计算过程如下。每个下标至多进出栈一次，时间、空间均为 O(L)。严格比较 > 保留最早起点。

核心代码 3-1 最长合法括号子串的计算

```
int start=0;
int max_len=0;
int len;
stack<int> st;
st.push(-1);
for(int i=0; i<str.size(); i++) {
  if(str[i]=='(') {
    st.push(i);
  } else {
    st.pop();
    if(st.empty()) {
      st.push(i);
    } else {
      len=i-st.top();
      if(len>max_len) {
        max_len=len;
        start=st.top()+1;
      }
    }
  }
}
cout<<max_len<<" "<<start<<endl;
```

2.3.5 调试分析

原来用 char 存下标，大下标被截断，改为 int。start 与边界分别初始化为0和-1。空文件未进入 while(getline(...))，改为读取一次后输出 0 0。随机测试可用题面 125_data.cpp；另复验空文件、空行、全左括号、同长起点、拼接和10万字符输入，6项正确。

2.3.6 总结和体会

答案起点和计算边界应分别维护；输入只有一个对象，读取失败时仍需得到空串答案。

2.4 队列的应用

2.4.1 问题描述

在0、1矩阵中，把通过上下左右相邻的1连通的格子视为一块，统计有效区域。依据题目所给 p44.cpp 的判定，只有整块完全处在最外圈时才排除；只要包含一个内部格子就计数，即使它与边界连通。

2.4.2 基本要求

读入行数 n、列数 m 和矩阵，0≤n,m≤1000。斜对角接触不算连通，每一块只统计一次。零行、零列时输出0。

2.4.3 数据结构设计

matrix 使用 vector<vector<int>>，checklist 保存待检查的行列坐标。格子入队时立即改成0，避免重复入队；isedge 持续记录整块是否全部在边界上。

2.4.4 功能说明

check() 检查坐标是否合法，checkedge() 检查是否位于最外圈。下面保留两个函数和 main() 中一整块区域的 BFS，省略矩阵输入和外层遍历。时间 O(nm)，空间 O(nm)。

核心代码 4-1 坐标合法性与边界判断

```
bool checkedge(int i, int k, int n, int m) {
  if(i==0||k==0||i==n-1||k==m-1) {
    return true;
  }
  return false;
}
bool check(int i, int k, int n, int m) {
  return (i<=n-1 && i>=0 && k>=0 && k<=m-1);
}
```

核心代码 4-2 从一个尚未访问的1开始搜索整块区域

```
bool isedge=true;
queue<pair<int, int>> checklist;
matrix[i][k]=0;
isedge=isedge&&checkedge(i, k, n, m);
checklist.push({i,k});
while(!checklist.empty()) {
  int x=checklist.front().first;
  int y=checklist.front().second;
  checklist.pop();
```

核心代码 4-2 续

```
  if(check(x-1, y, n, m)) {
    if(matrix[x-1][y]==1) {
      isedge=isedge&&checkedge(x-1, y, n, m);
      checklist.push({x-1, y});
      matrix[x-1][y]=0;
    }
  }
  if(check(x+1, y, n, m)) {
    if(matrix[x+1][y]==1) {
      isedge=isedge&&checkedge(x+1, y, n, m);
      checklist.push({x+1, y});
      matrix[x+1][y]=0;
    }
  }
  if(check(x, y-1, n, m)) {
    if(matrix[x][y-1]==1) {
      isedge=isedge&&checkedge(x, y-1, n, m);
      checklist.push({x, y-1});
      matrix[x][y-1]=0;
    }
  }
```

核心代码 4-2 续

```
  if(check(x, y+1, n, m)) {
    if(matrix[x][y+1]==1) {
      isedge=isedge&&checkedge(x, y+1, n, m);
      checklist.push({x, y+1});
      matrix[x][y+1]=0;
    }
  }
}
if(!isedge) {
  res+=1;
}
```

2.4.5 调试分析

局部变长数组曾在1000×1000时栈溢出，改为 vector。区域定义也曾误解为“碰边界就排除”；对照 p44.cpp 的 bo |= dir==4，确认应排除的是全在边界上的块。修改 isedge 初值及五处逻辑与。以所给文件保留的 Solution 类为参考，对100组补充随机矩阵和1组区分题意的小矩阵比对，101项一致。

2.4.6 总结和体会

队列管理已发现但未处理的格子，入队即标记可避免重复。样例通过只能说明部分行为正确，存在歧义时应核对参考程序，不能用同一误解构造测试后据此断言算法符合题意。

2.5 队列中的最大值问题

2.5.1 问题描述

设计容量为 n 的队列，支持 enqueue 入队、dequeue 出队、max 查询最大值以及 quit 输出剩余元素。目标是在保持先进先出的同时，尽可能降低最大值查询开销。

2.5.2 基本要求

容量上限10000，元素可正可负。空队列执行出队或 max 时输出 Queue is Empty，满队列入队时输出 Queue is Full，成功入队不输出。quit 按队首至队尾输出全部剩余元素，数值间一个空格、末尾换行。

2.5.3 数据结构设计

res 保存所有元素；seq 是非递增的最大值候选队列。新值淘汰尾部更小的候选，相等值保留。出队值等于 seq.front() 时同步删除一个；max 读取 seq.front()。

2.5.4 功能说明

操作在 main() 的命令分支中实现。下面摘录全部操作分支，省略头文件和输入输出加速设置。max、dequeue 为 O(1)；enqueue 单次最坏 O(n)、均摊 O(1)，K 条指令总时间 O(K)，空间 O(n)。

核心代码 5-1 容量控制与四种命令的处理

```
while(cin>>str) {
  if(str=="dequeue") {
    if(res.empty()) {
      cout<<"Queue is Empty"<<endl;
    } else {
      if(!seq.empty()&&seq.front()==res.front()) {
        seq.pop_front();
      }
      cout<<res.front()<<endl;
      res.pop();
    }
  } else if(str=="enqueue") {
    int el;
    cin>>el;
    if(res.size()==n) {
      cout<<"Queue is Full"<<endl;
    } else {
      while(!seq.empty()&&seq.back()<el) {
        seq.pop_back();
      }
      seq.push_back(el);
      res.push(el);
    }
```

核心代码 5-1 续

```
  } else if(str=="max") {
    if(res.empty()) {
      cout<<"Queue is Empty"<<endl;
    } else {
      cout<<seq.front()<<endl;
    }
  } else if(str=="quit") {
    bool first=true;
    while(!res.empty()) {
      if(!first) {
        cout<<" ";
      }
      first=false;
      cout<<res.front();
      res.pop();
    }
    cout<<endl;
    break;
  }
}
```

2.5.5 调试分析

使用题目提供的 p101.py，设置 NumPy 随机种子20261018后运行未修改的脚本，生成 input1.txt 至 input10.txt 及对应标准答案。逐个运行最终 C++ 程序，将完整输出与 output 文件逐字比较，10组全部一致。下表统计指令数时包含最后的 quit。

生成测试编号	队列容量	每组指令数范围	结果
1—2	100	1075—1080	2/2一致
3—4	6000	64387—65876	2/2一致
5—10	10000	101765—107885	6/6一致

重点检查重复最大值、满队列拒绝入队和 quit 的空格格式。最终代码的完整输出与10个标准答案文件逐字一致。已关闭输入输出同步；后续可将 endl 换成换行字符，减少强制刷新。

2.5.6 总结和体会

辅助队列保存的是可能成为最大值的候选，不是完整队列的复制。较大且较晚入队的元素会淘汰较小候选；使用严格小于号，才能让重复最大值与出队次数对应。

3 实验总结

本次实验完成了五道栈和队列应用题。列车进站和布尔表达式两题练习了栈的基本操作及执行顺序；最长括号子串通过保存下标计算长度和起点；矩阵区域计数使用队列遍历连通块；最大值队列增加了辅助双端队列，使最大值查询不必遍历全部元素。

调试时，我发现错误不仅来自计算过程，也来自状态和边界处理。列车题缺少失败出口会死循环；表达式题把两个栈混用会造成空栈访问；括号下标用 char 保存会在大数据时出错。修改后，我逐项检查了循环是否推进、访问栈顶前的状态，以及下标类型的范围。

矩阵题需要特别核对区域的定义。参考程序只排除全部位于边界上的连通块，并不是排除所有与边界相连的块。确认这一条件后，再根据格子是否在边界上更新标志，结果才与参考程序一致。这个问题说明，仅通过样例还不足以确认对题意的理解。

在效率方面，每个格子最多入队一次，因此 BFS 的总时间是 O(nm)。最大值队列虽然可能在一次入队中删除多个候选，但每个候选最多被删除一次，入队的均摊时间为 O(1)。局部矩阵改用 vector 解决了调用栈容量不足，空间复杂度仍为 O(nm)。

后续编写同类程序时，我会先确认输入组数、空输入、重复元素和输出分隔，再测试最大数据规模。随机测试使用题目给出的生成程序或参考判定，并保留输入、标准答案和实际输出，便于定位不一致的具体操作。

调试记录与复验方法

第五题使用 p101.py 生成10组输入和标准答案。设置 NumPy 随机种子20261018后运行原脚本，逐组将 C++ 输出与对应 output 文件比较。第三、四题题面给出的是 C++ 生成文件，按各题实际文件处理，不将补充测试记为官方生成文件的原始数据。

```
g++ -std=c++17 -O2 -D_GLIBCXX_ASSERTIONS hw2_5.cpp -o hw2_5.exe
python p101.py
hw2_5.exe < input1.txt > actual1.txt
```

以上重定向在 Windows 的 cmd 中执行，PowerShell 可用 cmd /c 包装。p101.py 需要 NumPy；固定种子须在同一个 Python 进程中、执行脚本前设置。